v1.0.1 Safety Baseline Implementation Plan

> **Resolution (2026-07-24, #54):** This plan shipped in full as release v1.0.1 (tag `v1.0.1`) via PRs #47 (commits f00a529/c07ff28), #48 (3f7d0a3), #49 (92b6e7d), #50 (bd9b3c2/ff2f78c), and release PR #51 (474744b), with the roadmap tick landing via PR #53. Every checklist item below is resolved: checked with evidence, migrated to a live issue, or rejected with rationale.
>
> **For agentic workers:** REQUIRED SUB-SKILL: Use superpowers:subagent-driven-development (recommended) or superpowers:executing-plans to implement this plan task-by-task. Steps use checkbox (`- [ ]`) syntax for tracking.

Goal: Make the read-only promise true at every output path (milestone v1.0.1, issues #3, #36, #37, #5, #38) and publish the first safe tag.

Architecture: One new shared output-safety module (`src/outpath.rs`) provides path identity (dev/ino), a protected-input set, symlink/containment gating, same-directory staging, and no-clobber/replace promotion. Index creation, dedup report writes, and master open all route through it — no feature keeps its own weaker copy. A second new module (`src/textsafe.rs`) centralizes terminal sanitizing; raw path bytes are captured at ingestion into new nullable BLOB columns and surfaced as hex sibling fields in JSON.

Tech Stack: Rust 2021, rusqlite (bundled SQLite), tar, anyhow, serde_json; tests via std::process + tempfile only (no new dev-dependencies).

Global Constraints

- **Safety invariant (permanent): BackupSage never rewrites or deletes from an archive.** No code path opens an archive with write access.
- v1.0.1 adds **no overwrite flag** — "unless a future explicit overwrite contract permits it" is explicitly future (spec line 71).
- No journals, no plan formats, no quarantine, no new analysis commands (deferred to v1.1/v1.2).
- Exit-code contract is script-stable: `0 ok · 1 error · 2 completed with skipped archives` (src/main.rs:3-4). Safety rejections are ordinary errors → exit 1.
- JSON report contract v1 is add-fields-only (src/report.rs:1-3); tests/dedup.rs:288 and tests/cli.rs pin existing names/values.
- Tests: std::process::Command + `env!("CARGO_BIN_EXE_backupsage")`, `run_ok/bin` helpers, `status.code() == Some(N)`, serde_json::Value probing, tempdir-pinned `--master`. Only dev-dependency is `tempfile`.
- Commits: `git -c user.name=claude_2010 -c user.email=262510778+tom2025b@users.noreply.github.com commit …`
- Branch → PR → merge to main; **never delete any branch**.

- Each parent issue spanning >1 independently testable change gets linked child issues before code (#3 and #5 qualify).
- CI runs `cargo build/test/clippy -D warnings/fmt` all `--locked` (.github/workflows/ci.yml).
- Unix-only identity primitives are acceptable (CI is ubuntu; the box is Linux). Guard with `#[cfg(unix)]` where std requires it.

PR / branch map

PR	Branch	Issues	Tasks
1	`v1.0.1-index-safety`	#3 (+ child issues)	1–3
2	`v1.0.1-report-noclobber`	#36	4
3	`v1.0.1-master-identity`	#37	5
4	`v1.0.1-sanitize-paths`	#5 (+ child issues)	6–7
5	`v1.0.1-release`	#38	8

Merge order: PR1 → PR2 → PR3 → PR4 → PR5 (PR2/PR3/PR4 depend on `outpath` from PR1; PR5 depends on all).

Task 0: Create child issues (tracking, no code)

Files: none (GitHub only).

Interfaces: Produces issue numbers referenced by later commit messages.

- [x] **Step 1: Create child issues for #3 and #5** — evidence: child issues #43/#44/#45/#46 created (all now closed), implemented in PR #47 / PR #50

```
gh issue create --title "Output-safety boundary: path identity, protected set,
staging, promotion" \
  --milestone "v1.0.1 — Safety baseline" \
  --body "Child of #3. New src/outpath.rs: FileId (dev/ino), ProtectedSet (files,
DB sidecars, dir trees), check_dest gating (symlink, hardlink, identity,
containment), same-directory staging, no-clobber and replace promotion. Unit-tested
in isolation. Parent: #3."
gh issue create --title "Index destinations: gate, same-source ownership, temp
build, promote-on-complete" \
  --milestone "v1.0.1 — Safety baseline" \
  --body "Child of #3. Route index creation through the output-safety boundary;
verify same-source ownership before any replacement; build in a same-directory temp
DB; promote only a completed index; make the CWD fallback pass the same gate.
Parent: #3."
gh issue create --title "Central terminal sanitizer for untrusted text" \
  --milestone "v1.0.1 — Safety baseline" \
  --body "Child of #5. New src/textsafe.rs; route paths, link targets, snippets,
labels, progress, warnings, errors through it. Terminal fixtures emit no raw C0/C1/
DEL/ESC/CSI/OSC. Parent: #5."
gh issue create --title "Lossless raw path bytes in index schema and JSON" \
  --milestone "v1.0.1 — Safety baseline" \
  --body "Child of #5. Capture raw path/link-target bytes at ingestion (tar
path_bytes/link_name_bytes, dir OsStr bytes) into nullable BLOB columns; emit hex
*_bytes sibling fields in JSON only when display is lossy. Parent: #5."
```

- [x] **Step 2: Link children in parents** — add a comment on #3 and #5 listing their child issue numbers. — evidence: verified comments on #3 (lists #43/#44) and #5 (lists #45/#46); children shipped in PR #47 / PR #50

Task 1: Output-safety boundary module (`src/outpath.rs`)

Files: - Create: `src/outpath.rs` - Modify: `src/lib.rs` (add `pub mod outpath;`) - Test: unit tests inside `src/outpath.rs` (`#[cfg(test)]`)

Interfaces: - Produces (used by Tasks 2, 4, 5): - `outpath::FileId` — `Copy + Eq`; `outpath::file_id(&Path) -> Option<FileId>` - `outpath::ProtectedSet::{new, add_file(&Path), add_db(&Path), add_dir_tree(&Path), check_dest(&Path) -> Result<()>, check_db_dest(&Path) -> Result<()>}` - `outpath::sidecar(&Path, &str) -> PathBuf` - `outpath::stage_path(&Path) -> PathBuf` - `outpath::promote_no_clobber(staged: &Path, final_path: &Path) -> Result<()>` - `outpath::promote_replace(staged: &Path, final_path: &Path) -> Result<()>` - `outpath::write_new_file(dest: &Path, bytes: &[u8], protected: &ProtectedSet) -> Result<()>`

- [x] **Step 1: Write failing unit tests** (in `src/outpath.rs` `#[cfg(test)] mod tests`, using `tempfile::tempdir()` — add `tempfile` is dev-dep already): — evidence: commit f00a529 (PR #47)

```
#[cfg(test)]
mod tests {
    use super::*;
    use std::fs;

    #[test]
    fn identity_matches_across_spellings_and_hardlinks() {
        let d = tempfile::tempdir().unwrap();
        let a = d.path().join("a.tar");
        fs::write(&a, b"data").unwrap();
        let alias = d.path().join("sub/../a.tar");
        fs::create_dir(d.path().join("sub")).unwrap();
        let hard = d.path().join("h.tar");
        fs::hard_link(&a, &hard).unwrap();
        let id = file_id(&a).unwrap();
        assert_eq!(file_id(&alias).unwrap(), id);
        assert_eq!(file_id(&hard).unwrap(), id);
    }

    #[test]
    fn check_dest_rejects_protected_file_symlink_and_containment() {
        let d = tempfile::tempdir().unwrap();
        let archive = d.path().join("a.tar");
        fs::write(&archive, b"data").unwrap();
        let srcdir = d.path().join("src");
        fs::create_dir(&srcdir).unwrap();
        fs::write(srcdir.join("member.txt"), b"m").unwrap();
        let mut p = ProtectedSet::new();
        p.add_file(&archive);
        p.add_dir_tree(&srcdir);
        // direct identity
        assert!(p.check_dest(&archive).is_err());
        // dot-dot spelling of the same file
        assert!(p.check_dest(&d.path().join("src/../a.tar")).is_err());
        // hardlink alias
        let hard = d.path().join("h.tar");
        fs::hard_link(&archive, &hard).unwrap();
        assert!(p.check_dest(&hard).is_err());
        // symlink at destination (even dangling) is refused
        let link = d.path().join("link.db");
        std::os::unix::fs::symlink(d.path().join("nowhere"), &link).unwrap();
        assert!(p.check_dest(&link).is_err());
        // containment in a protected tree
        assert!(p.check_dest(&srcdir.join("out.db")).is_err());
        // unrelated new path is fine
        assert!(p.check_dest(&d.path().join("fresh.db")).is_ok());
    }

    #[test]
    fn check_db_dest_covers_sidecars() {
        let d = tempfile::tempdir().unwrap();
        let victim = d.path().join("x.db-wal");
```

```
        fs::write(&victim, b"wal").unwrap();
        let mut p = ProtectedSet::new();
        p.add_file(&victim);
        assert!(p.check_db_dest(&d.path().join("x.db")).is_err());
        assert!(p.check_db_dest(&d.path().join("y.db")).is_ok());
    }

    #[test]
    fn write_new_file_never_clobbers_and_stages_invisibly() {
        let d = tempfile::tempdir().unwrap();
        let dest = d.path().join("report.json");
        let p = ProtectedSet::new();
        write_new_file(&dest, b"{}", &p).unwrap();
        assert_eq!(fs::read(&dest).unwrap(), b"{}");
        // second write to the same path fails and leaves content intact
        assert!(write_new_file(&dest, b"XX", &p).is_err());
        assert_eq!(fs::read(&dest).unwrap(), b"{}");
        // no staging debris
        let names: Vec<_> = fs::read_dir(d.path()).unwrap()
            .map(|e| e.unwrap().file_name().into_string().unwrap()).collect();
        assert!(names.iter().all(|n| !n.contains(".tmp.")), "{names:?}");
        // dangling symlink destination is refused, symlink left in place
        let link = d.path().join("link.json");
        std::os::unix::fs::symlink(d.path().join("gone"), &link).unwrap();
        assert!(write_new_file(&link, b"x", &p).is_err());
        assert!(fs::symlink_metadata(&link).unwrap().file_type().is_symlink());
    }

    #[test]
    fn promote_replace_swaps_atomically() {
        let d = tempfile::tempdir().unwrap();
        let fin = d.path().join("i.db");
        fs::write(&fin, b"old").unwrap();
        let staged = stage_path(&fin);
        fs::write(&staged, b"new").unwrap();
        promote_replace(&staged, &fin).unwrap();
        assert_eq!(fs::read(&fin).unwrap(), b"new");
        assert!(!staged.exists());
    }
}
```

- [x] **Step 2: Run tests, verify they fail** — `cargo test outpath` → FAIL (module missing). — evidence: commit f00a529 (PR #47)
- [x] **Step 3: Implement `src/outpath.rs`:** — evidence: commit f00a529 (PR #47)

```rust
//! Output-path safety boundary (v1.0.1).
//!
//! Every place BackupSage creates or replaces a file goes through this
//! module: path identity, protected inputs, no-clobber promotion, and
//! precondition checks. No feature implements its own weaker copy.

use anyhow::{bail, Context, Result};
use std::fs;
use std::path::{Path, PathBuf};

/// Identity of an existing filesystem object (device + inode). Two
/// paths with equal `FileId`s are the same file regardless of spelling,
/// symlinks, or hardlinks.
#[derive(Clone, Copy, PartialEq, Eq, Debug)]
pub struct FileId {
    dev: u64,
    ino: u64,
}

/// Identity of whatever `path` resolves to (follows symlinks), or None
/// if nothing exists there.
pub fn file_id(path: &Path) -> Option<FileId> {
    use std::os::unix::fs::MetadataExt;
    let md = fs::metadata(path).ok()?;
    Some(FileId { dev: md.dev(), ino: md.ino() })
}

/// Paths a command must never write over: source archives, directory
/// sources (whole trees), input indexes, the master catalog, and the
/// SQLite sidecars of every protected database.
#[derive(Default)]
pub struct ProtectedSet {
    files: Vec<(FileId, PathBuf)>,
    dirs: Vec<PathBuf>,
}

impl ProtectedSet {
    pub fn new() -> Self {
        Self::default()
    }

    /// Protect the file `path` currently resolves to, if any.
    pub fn add_file(&mut self, path: &Path) {
        if let Some(id) = file_id(path) {
            self.files.push((id, path.to_path_buf()));
        }
    }

    /// Protect a SQLite database plus its `-wal`/`-shm` sidecars.
    pub fn add_db(&mut self, path: &Path) {
        self.add_file(path);
        self.add_file(&sidecar(path, "-wal"));
```

```
        self.add_file(&sidecar(path, "-shm"));
    }

    /// Protect an entire directory tree.
    pub fn add_dir_tree(&mut self, path: &Path) {
        if let Ok(canon) = path.canonicalize() {
            self.dirs.push(canon);
        }
    }

    /// Fail unless a single file may be created at `dest`.
    pub fn check_dest(&self, dest: &Path) -> Result<()> {
        self.check_one(dest)
    }

    /// Fail unless a SQLite database may be created at `dest` — also
    /// gates the `-wal`/`-shm` names SQLite will create beside it.
    pub fn check_db_dest(&self, dest: &Path) -> Result<()> {
        self.check_one(dest)?;
        self.check_one(&sidecar(dest, "-wal"))?;
        self.check_one(&sidecar(dest, "-shm"))
    }

    fn check_one(&self, dest: &Path) -> Result<()> {
        let name = dest
            .file_name()
            .with_context(|| format!("output path '{}' has no file name",
dest.display()))?;
        // A symlink at the destination — dangling or not — would make the
        // write land somewhere else. Always refuse.
        if let Ok(md) = fs::symlink_metadata(dest) {
            if md.file_type().is_symlink() {
                bail!("refusing to write through symlink '{}'", dest.display());
            }
        }
        // Canonicalize the parent so `sub/../x` spellings and symlinked
        // directories cannot dodge the checks below.
        let parent = match dest.parent() {
            Some(p) if !p.as_os_str().is_empty() => p.to_path_buf(),
            _ => PathBuf::from("."),
        };
        let canon_parent = parent.canonicalize().with_context(|| {
            format!("output directory '{}' is not accessible", parent.display())
        })?;
        let resolved = canon_parent.join(name);
        if let Some(id) = file_id(&resolved) {
            if let Some((_, hit)) = self.files.iter().find(|(fid, _)| *fid == id) {
                bail!(
                    "output '{}' is protected input '{}' (same file)",
                    dest.display(),
                    hit.display()
                );
            }
```

```
        }
        for root in &self.dirs {
            if resolved.starts_with(root) {
                bail!(
                    "output '{}' is inside protected source directory '{}'",
                    dest.display(),
                    root.display()
                );
            }
        }
        Ok(())
    }
}

/// SQLite sidecar naming: `x.db` → `x.db-wal` / `x.db-shm`.
pub fn sidecar(path: &Path, suffix: &str) -> PathBuf {
    let mut name = path.file_name().unwrap_or_default().to_os_string();
    name.push(suffix);
    path.with_file_name(name)
}

/// Same-directory staging name for building `final_path`:
/// `dir/x.db` → `dir/.x.db.tmp.<pid>`.
pub fn stage_path(final_path: &Path) -> PathBuf {
    let mut name = std::ffi::OsString::from(".");
    name.push(final_path.file_name().unwrap_or_default());
    name.push(format!(".tmp.{}", std::process::id()));
    final_path.with_file_name(name)
}

/// Publish `staged` at `final_path` without replacing anything:
/// `hard_link` fails if any entry exists at `final_path` and never
/// follows a symlink there. The staged name is removed afterwards.
pub fn promote_no_clobber(staged: &Path, final_path: &Path) -> Result<()> {
    fs::hard_link(staged, final_path).with_context(|| {
        format!(
            "output '{}' already exists (BackupSage never overwrites)",
            final_path.display()
        )
    })?;
    let _ = fs::remove_file(staged);
    Ok(())
}

/// Replace `final_path` with `staged` in one rename. Callers must have
/// verified the file being replaced is theirs to replace.
pub fn promote_replace(staged: &Path, final_path: &Path) -> Result<()> {
    fs::rename(staged, final_path).with_context(|| {
        format!("could not promote staged output to '{}'", final_path.display())
    })
}

/// Create a brand-new file at `dest` with `bytes`: gate against the
```

```
/// protected set, stage in the destination directory, then promote
/// no-clobber. Failure at any point leaves no visible partial output.
pub fn write_new_file(dest: &Path, bytes: &[u8], protected: &ProtectedSet) ->
Result<()> {
    protected.check_dest(dest)?;
    if fs::symlink_metadata(dest).is_ok() {
        bail!(
            "output '{}' already exists (BackupSage never overwrites)",
            dest.display()
        );
    }
    let staged = stage_path(dest);
    let _ = fs::remove_file(&staged); // our own crashed-run debris only
    let write = (|| -> Result<()> {
        use std::io::Write;
        let mut f = fs::OpenOptions::new()
            .write(true)
            .create_new(true)
            .open(&staged)?;
        f.write_all(bytes)?;
        f.sync_all()?;
        Ok(())
    })();
    if let Err(e) = write {
        let _ = fs::remove_file(&staged);
        return Err(e).with_context(|| format!("writing '{}'", dest.display()));
    }
    if let Err(e) = promote_no_clobber(&staged, dest) {
        let _ = fs::remove_file(&staged);
        return Err(e);
    }
    Ok(())
}
```

And in `src/lib.rs` add `pub mod outpath;` alongside the existing modules.

- [x] **Step 4: Run tests** — `cargo test outpath` → all PASS. `cargo clippy --all-targets -- -D warnings` clean. — evidence: PR #47 CI green / commit f00a529
- [x] **Step 5: Commit** — `feat: output-safety boundary — path identity, protected set, staging, promotion (child of #3)` — evidence: commit f00a529

Task 2: Index destinations — gate, ownership, temp build, promote

Files: - Modify: `src/store.rs` (create_v3: remove blind deletes, require fresh path) - Modify: `src/indexer.rs` (create_db_with_fallback → gated staging; IndexRun promote on finish; run_index protected-set assembly) - Modify: `src/source_dir.rs` (own-output skip covers staged name) - Test: `tests/safety.rs` (new), `tests/common/mod.rs` (digest/inode helpers)

Interfaces: - Consumes: everything from Task 1. - Produces: - `store::create_v3` now FAILS if the path exists (no deletes). Callers always hand it a fresh staged path. - `indexer::IndexPaths { staged: PathBuf, final_path: PathBuf }` internal struct; `IndexRun::finish` performs `fold_wal` → close → `promote_replace` → removal of prior same-source sidecar debris. - `indexer::assert_replaceable_index(dest: &Path, source: &Path, source_type: &str) -> Result<()>` — ownership probe. - Behavior contract for tests: unsafe destination → exit 1, message starts `error:`, protected file digest+inode unchanged.

- [x] **Step 1: Add fixture helpers to `tests/common/mod.rs`:** — evidence: commit c07ff28 (PR #47)

```
/// BLAKE3 hex digest of a file's bytes.
pub fn digest_of(path: &Path) -> String {
    blake3::hash(&std::fs::read(path).unwrap()).to_hex().to_string()
}

/// Inode number (identity) of the entry at `path` itself (no follow).
pub fn inode_of(path: &Path) -> u64 {
    use std::os::unix::fs::MetadataExt;
    std::fs::symlink_metadata(path).unwrap().ino()
}
```

- [x] **Step 2: Write failing integration tests `tests/safety.rs`** (std::process conventions from tests/cli.rs — copy `bin()`, `run()` helpers locally): — evidence: commit c07ff28 (PR #47)

```
//! v1.0.1 destructive-path regression fixtures: every attack leaves
//! protected fixture digests and identities unchanged.
mod common;
use common::*;
use std::fs;
use std::path::Path;
use std::process::{Command, Output};

fn bin() -> Command {
    Command::new(env!("CARGO_BIN_EXE_backupsage"))
}

fn run(args: &[&str], cwd: &Path) -> Output {
    bin().args(args).current_dir(cwd).output().unwrap()
}

#[test]
fn index_dest_at_archive_fails_closed() {
    let d = tempfile::tempdir().unwrap();
    let tar = write_archive(d.path(), "a.tar", &build_tar(&[("f.txt",
b"hi".to_vec())]));
    let (dig, ino) = (digest_of(&tar), inode_of(&tar));
    let out = run(&["index", "a.tar", "-i", "a.tar"], d.path());
    assert_eq!(out.status.code(), Some(1));
    assert_eq!((digest_of(&tar), inode_of(&tar)), (dig, ino));
}

#[test]
fn index_dest_symlink_and_hardlink_to_archive_fail_closed() {
    let d = tempfile::tempdir().unwrap();
    let tar = write_archive(d.path(), "a.tar", &build_tar(&[("f.txt",
b"hi".to_vec())]));
    let (dig, ino) = (digest_of(&tar), inode_of(&tar));
    std::os::unix::fs::symlink(&tar, d.path().join("s.db")).unwrap();
    fs::hard_link(&tar, d.path().join("h.db")).unwrap();
    for dest in ["s.db", "h.db"] {
        let out = run(&["index", "a.tar", "-i", dest], d.path());
        assert_eq!(out.status.code(), Some(1), "dest {dest}");
        assert_eq!((digest_of(&tar), inode_of(&tar)), (dig, ino), "dest {dest}");
    }
    // symlink still a symlink, not replaced
    assert!
(fs::symlink_metadata(d.path().join("s.db")).unwrap().file_type().is_symlink());
}

#[test]
fn index_dest_inside_source_dir_fails_closed() {
    let d = tempfile::tempdir().unwrap();
    let src = d.path().join("photos");
    fs::create_dir(&src).unwrap();
    fs::write(src.join("m.txt"), b"member").unwrap();
    let dig = digest_of(&src.join("m.txt"));
```

```
    let out = run(&["index", "photos", "-i", "photos/m.txt"], d.path());
    assert_eq!(out.status.code(), Some(1));
    assert_eq!(digest_of(&src.join("m.txt")), dig);
}

#[test]
fn index_dest_foreign_file_and_foreign_index_rejected_unchanged() {
    let d = tempfile::tempdir().unwrap();
    write_archive(d.path(), "a.tar", &build_tar(&[("f.txt", b"hi".to_vec())]));
    write_archive(d.path(), "b.tar", &build_tar(&[("g.txt", b"yo".to_vec())]));
    // foreign non-index file
    fs::write(d.path().join("notes.db"), b"not sqlite").unwrap();
    let dig = digest_of(&d.path().join("notes.db"));
    let out = run(&["index", "a.tar", "-i", "notes.db"], d.path());
    assert_eq!(out.status.code(), Some(1));
    assert_eq!(digest_of(&d.path().join("notes.db")), dig);
    // index owned by another source
    let ok = run(&["index", "b.tar"], d.path());
    assert!(ok.status.success());
    let bdig = digest_of(&d.path().join("b.tar.db"));
    let out = run(&["index", "a.tar", "-i", "b.tar.db"], d.path());
    assert_eq!(out.status.code(), Some(1));
    assert_eq!(digest_of(&d.path().join("b.tar.db")), bdig);
}

#[test]
fn interrupted_reindex_preserves_last_good_index() {
    let d = tempfile::tempdir().unwrap();
    let tar = write_archive(d.path(), "a.tar", &build_tar(&[("f.txt",
b"v1".to_vec())]));
    assert!(run(&["index", "a.tar"], d.path()).status.success());
    let good = digest_of(&d.path().join("a.tar.db"));
    // corrupt the archive → re-index fails mid-build
    fs::write(&tar, b"garbage that is not a tar").unwrap();
    let out = run(&["index", "a.tar"], d.path());
    assert_eq!(out.status.code(), Some(1));
    // prior completed index untouched and still searchable
    assert_eq!(digest_of(&d.path().join("a.tar.db")), good);
    let s = run(&["search", "v1", "--db", "a.tar.db"], d.path());
    assert!(s.status.success());
}

#[test]
fn distinct_dest_and_same_source_reindex_succeed() {
    let d = tempfile::tempdir().unwrap();
    write_archive(d.path(), "a.tar", &build_tar(&[("f.txt", b"one".to_vec())]));
    assert!(run(&["index", "a.tar", "-i", "custom.db"],
d.path()).status.success());
    // same-source re-index at the same dest replaces the old index
    assert!(run(&["index", "a.tar", "-i", "custom.db"],
d.path()).status.success());
    let s = run(&["search", "one", "--db", "custom.db"], d.path());
    assert!(s.status.success());
```

```
    // no staging debris
    let names: Vec<_> = fs::read_dir(d.path()).unwrap()
        .map(|e| e.unwrap().file_name().into_string().unwrap()).collect();
    assert!(names.iter().all(|n| !n.contains(".tmp.")), "{names:?}");
}

#[test]
fn cwd_fallback_refuses_foreign_db() {
    let d = tempfile::tempdir().unwrap();
    let ro = d.path().join("ro");
    fs::create_dir(&ro).unwrap();
    write_archive(&ro, "a.tar", &build_tar(&[("f.txt", b"hi".to_vec())]));
    // pre-plant a foreign file at the fallback name in CWD
    fs::write(d.path().join("a.tar.db"), b"unrelated").unwrap();
    let dig = digest_of(&d.path().join("a.tar.db"));
    // make preferred destination unwritable so the fallback engages
    let mut perm = fs::metadata(&ro).unwrap().permissions();
    use std::os::unix::fs::PermissionsExt;
    perm.set_mode(0o555);
    fs::set_permissions(&ro, perm.clone()).unwrap();
    let out = run(&["index", "ro/a.tar"], d.path());
    perm.set_mode(0o755);
    fs::set_permissions(&ro, perm).unwrap();
    assert_eq!(out.status.code(), Some(1));
    assert_eq!(digest_of(&d.path().join("a.tar.db")), dig);
}
```

- [x] **Step 3: Run tests, verify current failures** — `cargo test --test safety` → the attack tests FAIL today (archive destroyed etc.). Record which assertions fail. — evidence: commit c07ff28 (PR #47)
- [x] **Step 4: Implement.** — evidence: commit c07ff28 (PR #47)

`src/store.rs` — `create_v3` stops deleting; it now requires a fresh path:

```
// Replace the delete-stale block (store.rs:76-85) with:
    if db_path.exists() || fs::symlink_metadata(db_path).is_ok() {
        anyhow::bail!(
            "internal error: create_v3 called on existing path '{}'",
            db_path.display()
        );
    }
```

`src/indexer.rs` — replace `create_db_with_fallback` with a gated, staged version and thread the staged/final pair through `IndexRun`:

```
pub(crate) struct IndexPaths {
    pub staged: PathBuf,
    pub final_path: PathBuf,
}

/// Gate a candidate final destination, verify ownership of anything
/// already there, and create the staged database beside it.
fn create_staged_db(
    final_path: &Path,
    meta: &store::SourceMeta,
    protected: &outpath::ProtectedSet,
) -> Result<(Connection, IndexPaths)> {
    protected.check_db_dest(final_path)?;
    assert_replaceable_index(final_path, &meta.source, meta.source_type)?;
    let staged = outpath::stage_path(final_path);
    // Our own crashed-run debris only: the staged namespace is ours.
    for p in [staged.clone(), outpath::sidecar(&staged, "-wal"),
outpath::sidecar(&staged, "-shm")] {
        let _ = fs::remove_file(&p);
    }
    let conn = store::create_v3(&staged, meta)?;
    Ok((conn, IndexPaths { staged, final_path: final_path.to_path_buf() }))
}

/// If something exists at `dest` it must be a completed BackupSage
/// index of THIS source; otherwise refuse to replace it.
fn assert_replaceable_index(dest: &Path, source: &Path, source_type: &str) ->
Result<()> {
    if fs::symlink_metadata(dest).is_err() {
        return Ok(()); // nothing there — plain create
    }
    let conn = crate::searcher::open_index(dest).with_context(|| {
        format!(
            "existing file '{}' is not a BackupSage index; refusing to replace it",
            dest.display()
        )
    })?;
    let stored_type = crate::searcher::get_meta(&conn, "source_type")
        .unwrap_or_else(|| "tar".into());
    let stored = crate::searcher::get_meta(&conn, "source")
        .or_else(|| crate::searcher::get_meta(&conn, "archive"))
        .unwrap_or_default();
    let same = stored_type == source_type
        && match (Path::new(&stored).canonicalize(), source.canonicalize()) {
            (Ok(a), Ok(b)) => a == b,
            _ => false,
        };
    if !same {
        bail!(
            "existing index '{}' belongs to source '{}', not '{}'; refusing to
replace it",
            dest.display(),
```

```
            stored,
            source.display()
        );
    }
    Ok(())
}
```

`create_db_with_fallback` keeps its signature but builds the `ProtectedSet` (tar → `add_file(source)`; dir → `add_dir_tree(source)`) — callers pass it in from `run_index` — and on preferred-location failure applies the SAME `create_staged_db` gate to the CWD fallback path before using it (and prints the existing fallback notice).

`IndexRun` gains the `IndexPaths`; `finish` order: `finalize_v3` → `fold_wal` → drop connection → `outpath::promote_replace(&staged, &final_path)` → best-effort `remove_file` of `final_path`'s stale `-wal`/`-shm` (pre-validated by the ownership gate). On any earlier error the staged files are removed and the final path is never touched.

`src/source_dir.rs` — the own-output skip (source_dir.rs:72-86) must now skip the STAGED names (`.x.db.tmp.<pid>` + sidecars) since that's what exists during the walk; keep skipping the final names too.

- [x] **Step 5: Run the full suite** — `cargo test` → all pass, including existing tests/index_v3.rs, tests/roundtrip.rs (re-index flows now stage+promote), tests/cli.rs. `cargo clippy --all-targets -- -D warnings && cargo fmt --check`. — evidence: PR #47 CI green / commit c07ff28
- [x] **Step 6: Commit** — `fix: index destinations gated, same-source ownership verified, last-good index preserved via staged promote (child of #3)` — evidence: commit c07ff28

Task 3: PR for issue #3

- [x] **Step 1:** Push `v1.0.1-index-safety`; open PR titled `fix: protect index destinations and preserve the last-good index (#3)` with body: what/why, attack matrix covered, acceptance-criteria checklist from #3, `Closes #3` plus `Closes` lines for its two child issues. — evidence: PR #47
- [x] **Step 2:** CI green → merge (merge commit, branch kept). — evidence: PR #47 merged, merge commit a96d183

Task 4: Dedup report no-clobber (#36)

Files: - Modify: `src/main.rs` (Dedup arm: protected-set assembly + `write_new_file`) - Test: `tests/safety.rs` (extend)

Interfaces: - Consumes: `outpath::{ProtectedSet, write_new_file}`. - Protected at report-write time: master path (+sidecars), every `--db` input (+sidecars), every registered archive's `db_path` and `source_path` (from `master.list()`).

- [x] **Step 1: Write failing tests** (append to tests/safety.rs): — evidence: commit 3f7d0a3 (PR #48)

```
#[test]
fn dedup_output_never_clobbers() {
    let d = tempfile::tempdir().unwrap();
    write_archive(d.path(), "a.tar", &build_tar(&[("f.txt", b"same".to_vec()),
("g.txt", b"same".to_vec())]));
    assert!(run(&["index", "a.tar"], d.path()).status.success());
    let master = d.path().join("m.db");
    let m = master.to_str().unwrap();
    assert!(run(&["--master", m, "master", "add", "a.tar.db"],
d.path()).status.success());
    let mdig = digest_of(&master);
    let adig = digest_of(&d.path().join("a.tar"));
    let idig = digest_of(&d.path().join("a.tar.db"));

    // -o at master, index, archive, and an existing unrelated file
    fs::write(d.path().join("existing.txt"), b"keep me").unwrap();
    for dest in ["m.db", "a.tar.db", "a.tar", "existing.txt"] {
        let out = run(&["--master", m, "dedup", "-o", dest], d.path());
        assert_eq!(out.status.code(), Some(1), "dest {dest}");
    }
    assert_eq!(digest_of(&master), mdig);
    assert_eq!(digest_of(&d.path().join("a.tar")), adig);
    assert_eq!(digest_of(&d.path().join("a.tar.db")), idig);
    assert_eq!(fs::read(d.path().join("existing.txt")).unwrap(), b"keep me");

    // symlink and hardlink aliases to the master are rejected, not unlinked
    std::os::unix::fs::symlink(&master, d.path().join("alias.json")).unwrap();
    fs::hard_link(&master, d.path().join("hard.json")).unwrap();
    for dest in ["alias.json", "hard.json"] {
        let out = run(&["--master", m, "dedup", "-o", dest], d.path());
        assert_eq!(out.status.code(), Some(1), "dest {dest}");
    }
    assert!
(fs::symlink_metadata(d.path().join("alias.json")).unwrap().file_type().is_symlink(
));
    assert_eq!(digest_of(&master), mdig);

    // a fresh distinct output receives the complete report, no staging debris
    let out = run(&["--master", m, "dedup", "--json", "-o", "report.json"],
d.path());
    assert!(out.status.success());
    let report: serde_json::Value =

serde_json::from_str(&fs::read_to_string(d.path().join("report.json")).unwrap()).un
wrap();
    assert_eq!(report["version"], 1);
    let names: Vec<_> = fs::read_dir(d.path()).unwrap()
        .map(|e| e.unwrap().file_name().into_string().unwrap()).collect();
    assert!(names.iter().all(|n| !n.contains(".tmp.")), "{names:?}");
}
```

- [x] **Step 2: Run, verify failures** — `cargo test --test safety dedup_output_never_clobbers` → FAIL (fs::write clobbers today). — evidence: commit 3f7d0a3 (PR #48)
- [x] **Step 3: Implement** in the `Commands::Dedup` arm (main.rs:198-211): build the set, replace `std::fs::write`: — evidence: commit 3f7d0a3 (PR #48)

```
if let Some(path) = &args.output {
    let mut protected = backupsage::outpath::ProtectedSet::new();
    protected.add_db(&master_path);
    for db in &args.dbs {
        protected.add_db(db);
    }
    for a in m.list()? {
        protected.add_db(Path::new(&a.db_path));
        protected.add_file(Path::new(&a.source_path));
    }
    backupsage::outpath::write_new_file(path, rendered.as_bytes(), &protected)
        .context("dedup report not written")?;
}
```

(`m.list()` for the in-memory `--db` mode lists the ad-hoc attachments; both modes covered.)

- [x] **Step 4: Run full suite + clippy + fmt** — all green. — evidence: PR #48 CI green / commit 3f7d0a3
- [x] **Step 5: Commit + PR** — `fix: dedup report writes are staged, alias-aware, and never clobber (#36)`; body maps #36 acceptance criteria to tests; `Closes #36`. Merge after CI. — evidence: commit 3f7d0a3 / PR #48 merged (bff7db8)

Task 5: Master catalog identity gate (#37)

Files: - Modify: `src/master.rs` (probe + gated `open_at` + stamp) - Test: `tests/safety.rs` (extend)

Interfaces: - Produces: - `master::MASTER_APPLICATION_ID: u32 = 0x4253_4147` (`"BSAG"`) - `master::probe(path: &Path) -> Result<MasterProbe>` where `enum MasterProbe { Absent, SignedMaster, LegacyMaster, PerSourceIndex, Foreign }` - `open_at` behavior: Absent → create+stamp; Signed/Legacy → open RW, stamp if legacy, run DDL; everything else → error, file untouched, no sidecars.

- [x] **Step 1: Write failing tests:** — evidence: commit 92b6e7d (PR #49)

```
#[test]
fn master_open_validates_identity_before_write() {
    let d = tempfile::tempdir().unwrap();
    write_archive(d.path(), "a.tar", &build_tar(&[("f.txt", b"hi".to_vec())]));
    assert!(run(&["index", "a.tar"], d.path()).status.success());

    // 1. per-source index must not be adopted or contaminated
    let idx = d.path().join("a.tar.db");
    let idig = digest_of(&idx);
    let out = run(&["--master", "a.tar.db", "master", "add", "a.tar.db"],
d.path());
    assert_eq!(out.status.code(), Some(1));
    assert_eq!(digest_of(&idx), idig, "per-source index must stay byte-identical");
    assert!(!d.path().join("a.tar.db-wal").exists());
    assert!(!d.path().join("a.tar.db-shm").exists());

    // 2. arbitrary SQLite database rejected unchanged
    let alien = d.path().join("alien.db");
    let conn = rusqlite::Connection::open(&alien).unwrap();
    conn.execute_batch("CREATE TABLE t(x); INSERT INTO t VALUES (1);").unwrap();
    drop(conn);
    let adig = digest_of(&alien);
    let out = run(&["--master", "alien.db", "master", "add", "a.tar.db"],
d.path());
    assert_eq!(out.status.code(), Some(1));
    assert_eq!(digest_of(&alien), adig);
    assert!(!d.path().join("alien.db-wal").exists());

    // 3. archive, symlink, sidecar-name, empty-file aliases fail closed
    fs::write(d.path().join("empty.db"), b"").unwrap();
    std::os::unix::fs::symlink(d.path().join("a.tar"),
d.path().join("ml.db")).unwrap();
    fs::write(d.path().join("base.db"), b"x").unwrap();
    for m in ["a.tar", "ml.db", "base.db-wal", "empty.db"] {
        let out = run(&["--master", m, "master", "add", "a.tar.db"], d.path());
        assert_eq!(out.status.code(), Some(1), "master {m}");
    }
    assert_eq!(digest_of(&d.path().join("a.tar")),
digest_of(&d.path().join("a.tar")));

    // 4. absent safe path creates a signed catalog that reopens fine
    assert!(run(&["--master", "m.db", "master", "add", "a.tar.db"],
d.path()).status.success());
    let conn = rusqlite::Connection::open(d.path().join("m.db")).unwrap();
    let appid: u32 = conn.query_row("PRAGMA application_id", [], |r|
r.get(0)).unwrap();
    assert_eq!(appid, 0x4253_4147);
    drop(conn);
    assert!(run(&["--master", "m.db", "master", "list"],
d.path()).status.success());

    // 5. legacy unsigned master (application_id 0) is adopted and stamped
```

```
    let conn = rusqlite::Connection::open(d.path().join("m.db")).unwrap();
    conn.pragma_update(None, "application_id", 0).unwrap();
    drop(conn);
    assert!(run(&["--master", "m.db", "master", "list"],
d.path()).status.success());
    let conn = rusqlite::Connection::open(d.path().join("m.db")).unwrap();
    let appid: u32 = conn.query_row("PRAGMA application_id", [], |r|
r.get(0)).unwrap();
    assert_eq!(appid, 0x4253_4147);
}
```

- [x] **Step 2: Run, verify failures** — cases 1–3 FAIL today (contamination), 4–5 partially pass by accident; record. — evidence: commit 92b6e7d (PR #49)
- [x] **Step 3: Implement in `src/master.rs`:** — evidence: commit 92b6e7d (PR #49)

```
pub const MASTER_APPLICATION_ID: u32 = 0x4253_4147; // "BSAG"

pub enum MasterProbe {
    Absent,
    SignedMaster,
    LegacyMaster,
    PerSourceIndex,
    Foreign,
}

/// Read-only identity probe. Never creates the file, never writes,
/// never leaves sidecars.
pub fn probe(path: &Path) -> Result<MasterProbe> {
    let md = match fs::symlink_metadata(path) {
        Err(_) => return Ok(MasterProbe::Absent),
        Ok(md) => md,
    };
    if md.file_type().is_symlink() {
        bail!("master path '{}' is a symlink; refusing", path.display());
    }
    // sidecar-name alias: "<base>-wal"/"<base>-shm" beside an existing base
    if let Some(name) = path.file_name().and_then(|n| n.to_str()) {
        for suf in ["-wal", "-shm"] {
            if let Some(base) = name.strip_suffix(suf) {
                if path.with_file_name(base).exists() {
                    bail!("master path '{}' is a SQLite sidecar; refusing",
path.display());
                }
            }
        }
    }
    if md.len() == 0 {
        return Ok(MasterProbe::Foreign); // never adopt a pre-existing empty file
    }
    // SQLite magic, read-only
    let mut head = [0u8; 16];
    use std::io::Read;
    let n = fs::File::open(path)?.read(&mut head)?;
    if n < 16 || &head != b"SQLite format 3\0" {
        return Ok(MasterProbe::Foreign);
    }
    let conn = Connection::open_with_flags(path,
OpenFlags::SQLITE_OPEN_READ_ONLY)?;
    let appid: u32 = conn.query_row("PRAGMA application_id", [], |r| r.get(0))?;
    if appid == MASTER_APPLICATION_ID {
        return Ok(MasterProbe::SignedMaster);
    }
    let has = |name: &str| -> Result<bool> {
        Ok(conn
            .query_row(
                "SELECT 1 FROM sqlite_master WHERE name = ?1",
                [name],
```

```
                |_| Ok(()),
            )
            .optional()?
            .is_some())
    };
    if has("files_fts")? {
        return Ok(MasterProbe::PerSourceIndex);
    }
    if appid == 0 && has("archives")? && has("files")? {
        // v1.0.0-created master: archives + master-shaped files
        let pb0: bool = conn
            .prepare("SELECT 1 FROM pragma_table_info('files') WHERE name='pb0'")?
            .query_row([], |_| Ok(()))
            .optional()?
            .is_some();
        if pb0 {
            return Ok(MasterProbe::LegacyMaster);
        }
    }
    Ok(MasterProbe::Foreign)
}
```

`open_at` becomes:

```
pub fn open_at(path: &Path) -> Result<Master> {
    match probe(path)? {
        MasterProbe::Absent => {
            if let Some(parent) = path.parent().filter(|p| !
p.as_os_str().is_empty()) {
                fs::create_dir_all(parent)
                    .with_context(|| format!("creating '{}'", parent.display()))?;
            }
            let conn = Connection::open(path)?;
            conn.pragma_update(None, "application_id", MASTER_APPLICATION_ID)?;
            init_master_conn(&conn)?;
            Ok(Master { conn })
        }
        MasterProbe::SignedMaster | MasterProbe::LegacyMaster => {
            let conn = Connection::open(path)?;
            conn.pragma_update(None, "application_id", MASTER_APPLICATION_ID)?;
            init_master_conn(&conn)?; // supported migrations (idempotent DDL)
            Ok(Master { conn })
        }
        MasterProbe::PerSourceIndex => bail!(
            "'{}' is a per-source index, not a master catalog; refusing to modify
it",
            path.display()
        ),
        MasterProbe::Foreign => bail!(
            "'{}' exists but is not a BackupSage master catalog; refusing to modify
it",
            path.display()
        ),
    }
}
```

(`use rusqlite::OptionalExtension;` for `.optional()`.)

- [x] **Step 4: Full suite + clippy + fmt** — tests/master.rs must still pass (its masters are created by this code → signed). — evidence: PR #49 CI green / commit 92b6e7d
- [x] **Step 5: Commit + PR** — `fix: validate master catalog identity before read-write open (#37)`; `Closes #37`. Merge after CI. — evidence: commit 92b6e7d / PR #49 merged (7aca01d)

Task 6: Central terminal sanitizer (#5, child A)

Files: - Create: `src/textsafe.rs` - Modify: `src/lib.rs` (`pub mod textsafe;`) - Modify sinks: `src/main.rs` (error print :21, search table :694/:697, master table :629-635, federated headers/notes :78-87, dedup text report label/path/reason sites :499-604, inspect : 387-405, run_master prints :243-327), `src/indexer.rs` (progress :509, warns :168/:199), `src/source_dir.rs` (progress :96, warns :64/:142), `src/searcher.rs` (hint :289-292) - Test: `tests/safety.rs` (extend)

Interfaces: - Produces: `textsafe::sanitize(&str) -> Cow<'_, str>` — C0 → U+2400 control pictures, DEL → ␡ (U+2421), C1 → U+FFFD; everything else unchanged. - Rule:

sanitize the UNTRUSTED FIELD at the sink (table cell, `{}` interpolation site, progress message), not whole trusted format strings — except main.rs:21 where the whole rendered error chain is sanitized.

- [x] **Step 1: Write failing tests:** — evidence: commit bd9b3c2 (PR #50)

```
#[test]
fn terminal_output_contains_no_raw_controls() {
    let d = tempfile::tempdir().unwrap();
    // member names and content carrying ESC/CSI/OSC/C0
    let evil_name = "e\x1b]0;pwned\x07vil\x1b[31m.txt";
    let evil_content = b"BEL\x07 ESC\x1b[2J CSI txt searchable".to_vec();
    write_archive(d.path(), "a.tar", &build_tar(&[(evil_name, evil_content)]));
    assert!(run(&["index", "a.tar"], d.path()).status.success());
    let out = run(&["search", "searchable", "--db", "a.tar.db"], d.path());
    assert!(out.status.success());
    for stream in [&out.stdout, &out.stderr] {
        let s = String::from_utf8_lossy(stream);
        assert!(!s.chars().any(|c| matches!(c,
            '\u{0}'..='\u{8}' | '\u{b}'..='\u{1f}' | '\u{7f}'..='\u{9f}')),
            "raw control in output: {s:?}");
    }
    // inspect echoes the (sanitized) name and any link target
    let out = run(&["inspect", "--db", "a.tar.db"], d.path());
    let s = String::from_utf8_lossy(&out.stdout);
    assert!(!s.contains('\x1b') && !s.contains('\x07'), "{s:?}");
}

#[test]
fn ordinary_unicode_stays_readable() {
    let d = tempfile::tempdir().unwrap();
    write_archive(d.path(), "a.tar",
        &build_tar(&[("días/naïve-写真.txt", b"unicode fine".to_vec())]));
    assert!(run(&["index", "a.tar"], d.path()).status.success());
    let out = run(&["search", "unicode", "--db", "a.tar.db"], d.path());
    let s = String::from_utf8_lossy(&out.stdout);
    assert!(s.contains("días") && s.contains("写真"), "{s}");
}
```

- [x] **Step 2: Run, verify FAIL** (raw ESC in table today). — evidence: commit bd9b3c2 (PR #50)
- [x] **Step 3: Implement `src/textsafe.rs`:** — evidence: commit bd9b3c2 (PR #50; commit message enumerates every wired sink)

```
//! Central sanitizer for untrusted text headed to a terminal.
//!
//! Every untrusted field (archive member paths, link targets, snippets,
//! labels, progress messages, warnings, error chains) passes through
//! `sanitize` before printing. C0 controls map to the Unicode control
//! pictures (U+2400 block) so output stays honest and readable; C1
//! controls have no pictures and map to U+FFFD.

use std::borrow::Cow;

pub fn sanitize(s: &str) -> Cow<'_, str> {
    if !s.chars().any(is_risky) {
        return Cow::Borrowed(s);
    }
    Cow::Owned(s.chars().map(map_char).collect())
}

fn is_risky(c: char) -> bool {
    matches!(c, '\u{0}'..='\u{1f}' | '\u{7f}'..='\u{9f}')
}

fn map_char(c: char) -> char {
    match c {
        '\u{0}'..='\u{1f}' => char::from_u32(0x2400 + c as u32).unwrap(),
        '\u{7f}' => '\u{2421}',
        '\u{80}'..='\u{9f}' => '\u{FFFD}',
        _ => c,
    }
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn controls_are_pictured_del_and_c1_replaced() {
        assert_eq!(sanitize("a\x1b[31mb"), "a␛[31mb");
        assert_eq!(sanitize("x\x07y\x00z"), "x␇y␀z");
        assert_eq!(sanitize("d\x7fe"), "d␡e");
        assert_eq!(sanitize("c\u{9b}d"), "c\u{FFFD}d");
        assert_eq!(sanitize("newline\nkept-visible"), "newline␤kept-visible");
    }

    #[test]
    fn clean_text_is_borrowed_unchanged() {
        assert!(matches!(sanitize("días 写真 ok"), Cow::Borrowed(_)));
    }
}
```

Then wire every sink listed in **Files** — pattern per sink: `Cell::new(textsafe::sanitize(&hit.path))`,

`pb.set_message(textsafe::sanitize(&truncate_path(...)).into_owned())`, `eprintln!("error: {}", textsafe::sanitize(&format!("{e:#}")))`, etc. Grep-verify coverage: every `println!/eprintln!/Cell::new/set_message` whose argument can carry archive-derived text goes through `sanitize`.

- [x] **Step 4: Full suite + clippy + fmt.** — evidence: PR #50 CI green / commit bd9b3c2
- [x] **Step 5: Commit** — `fix: central terminal sanitizer for untrusted text (child of #5)` — evidence: commit bd9b3c2

Task 7: Lossless raw path bytes (#5, child B)

Files: - Modify: `src/store.rs` (schema: `path_raw BLOB`, `link_target_raw BLOB`; `EntryRecord` fields; insert) - Modify: `src/indexer.rs` (capture `entry.path_bytes()` / `entry.link_name_bytes()`) - Modify: `src/source_dir.rs` (capture OsStr bytes via `std::os::unix::ffi::OsStrExt`) - Modify: `src/searcher.rs` (`SearchHit.path_raw: Option<Vec<u8>>`, guarded SELECT) - Modify: `src/master.rs` (master `files.path_raw BLOB` column + guarded ALTER migration + replicate copies it) - Modify: `src/report.rs` (`Member.path_bytes: Option<String>` hex, skip-if-none), `src/dedup.rs` (populate), `src/main.rs` (search JSON `path_bytes`, inspect raw target) - Test: `tests/safety.rs` (extend), `tests/dedup.rs` (extend contract list)

Interfaces: - Consumes: Task 6's module (display strings stay lossy+sanitized; raw bytes ride beside). - Produces: - Capture rule: `std::str::from_utf8(bytes)` Ok → store text only, raw NULL; Err → store lossy text + raw BLOB. - JSON rule: `path_bytes` / `link_target_bytes` = lowercase hex of the raw bytes, present ONLY when raw was stored (display is lossy). Existing `path` field unchanged. - Old indexes (no columns): readers check `pragma_table_info` once per connection and treat missing columns as NULL. Schema_version stays 3; new meta key `path_raw = "1"` marks capable indexes. - Master migration: `ALTER TABLE files ADD COLUMN path_raw BLOB` guarded by pragma_table_info — this is a "supported migration" under #37's signed-master gate.

- [x] **Step 1: Write failing tests:** — evidence: commit ff2f78c (PR #50)

```
#[test]
fn json_round_trips_non_utf8_paths() {
    let d = tempfile::tempdir().unwrap();
    // invalid UTF-8 member name: "f" 0xFF 0xFE ".txt"
    let raw_name: &[u8] = b"f\xff\xfe.txt";
    let mut header = tar::Header::new_gnu();
    header.set_size(4);
    header.set_mode(0o644);
    header.set_mtime(1_700_000_001);
    let mut ar = tar::Builder::new(Vec::new());
    use std::os::unix::ffi::OsStrExt;
    let name = std::path::PathBuf::from(std::ffi::OsStr::from_bytes(raw_name));
    ar.append_data(&mut header, &name, &b"data"[..]).unwrap();
    let bytes = ar.into_inner().unwrap();
    write_archive(d.path(), "a.tar", &bytes);

    assert!(run(&["index", "a.tar"], d.path()).status.success());
    let out = run(&["search", "data", "--db", "a.tar.db", "--json"], d.path());
    assert!(out.status.success());
    let v: serde_json::Value =
        serde_json::from_str(&String::from_utf8_lossy(&out.stdout)).unwrap();
    let hit = &v["hits"][0];
    // display field lossy but present; raw bytes round-trip exactly
    assert!(hit["path"].as_str().unwrap().contains('\u{FFFD}'));
    let hex = hit["path_bytes"].as_str().unwrap();
    let decoded: Vec<u8> = (0..hex.len()).step_by(2)
        .map(|i| u8::from_str_radix(&hex[i..i + 2], 16).unwrap()).collect();
    assert_eq!(decoded, raw_name);
}

#[test]
fn clean_utf8_paths_emit_no_bytes_field() {
    let d = tempfile::tempdir().unwrap();
    write_archive(d.path(), "a.tar", &build_tar(&[("plain.txt",
b"data".to_vec())]));
    assert!(run(&["index", "a.tar"], d.path()).status.success());
    let out = run(&["search", "data", "--db", "a.tar.db", "--json"], d.path());
    let v: serde_json::Value =
        serde_json::from_str(&String::from_utf8_lossy(&out.stdout)).unwrap();
    assert!(v["hits"][0].get("path_bytes").is_none()
        || v["hits"][0]["path_bytes"].is_null());
    assert_eq!(v["hits"][0]["path"], "plain.txt");
}
```

Also extend `tests/dedup.rs::json_contract_field_names_are_stable`: existing keys unchanged (the new fields are optional siblings — presence-only test list stays valid; add a comment noting `path_bytes` is optional).

- [x] **Step 2: Run, verify FAIL** (`path_bytes` absent; today the raw bytes are unrecoverable). — evidence: commit ff2f78c (PR #50)
- [x] **Step 3: Implement** per the Interfaces block: — evidence: commit ff2f78c (PR #50)

Capture (indexer.rs, replacing :502-505 and :515-520):

```
let raw = entry.path_bytes();
let (entry_path, path_raw): (String, Option<Vec<u8>>) = match
std::str::from_utf8(&raw) {
    Ok(s) => (s.to_owned(), None),
    Err(_) => (String::from_utf8_lossy(&raw).into_owned(), Some(raw.to_vec())),
};
let link = entry.link_name_bytes().map(|b| match std::str::from_utf8(&b) {
    Ok(s) => (s.to_owned(), None),
    Err(_) => (String::from_utf8_lossy(&b).into_owned(), Some(b.to_vec())),
});
```

Hex helper (in report.rs, shared):

```
pub fn to_hex(bytes: &[u8]) -> String {
    bytes.iter().map(|b| format!("{b:02x}")).collect()
}
```

`Member` gains:

```
    #[serde(skip_serializing_if = "Option::is_none")]
    pub path_bytes: Option<String>,
```

Schema (store.rs create_v3 files table): add `path_raw BLOB` and `link_target_raw BLOB` columns; `EntryRecord` gains `path_raw: Option<&'a [u8]>, link_target_raw: Option<&'a [u8]>`; insert binds them; meta gains `path_raw = "1"`. Reader guard (searcher.rs + master.rs replicate):

```
fn has_column(conn: &Connection, table: &str, col: &str) -> bool {
    conn.prepare(&format!("SELECT 1 FROM pragma_table_info('{table}') WHERE name=?
1"))
        .and_then(|mut s| s.query_row([col], |_| Ok(())).optional())
        .ok().flatten().is_some()
}
```

- [x] **Step 4: Full suite + clippy + fmt.** Watch tests/cli.rs pinned values — `path` display fields unchanged, so they pass. — evidence: PR #50 CI green / commit ff2f78c
- [x] **Step 5: Commit + PR** — `fix: sanitize untrusted text and preserve lossless raw path bytes (#5)`; PR body maps the four #5 acceptance criteria to the new tests; `Closes #5` + its two children. Merge after CI. — evidence: commit ff2f78c / PR #50 merged (491e934)

Task 8: Advisories, release gates, v1.0.1 (#38)

Files: - Modify: `Cargo.toml` (version = "1.0.1"), `Cargo.lock` (cargo update) - Create: `docs/SECURITY.md` (advisory disposition) - Modify: `.github/workflows/ci.yml` (audit job) - Modify:

`docs/ROADMAP.md` (tick v1.0.1 checkboxes) - Create: `docs/adr/0001-output-safety-boundary.md`, `docs/adr/0002-master-signature-and-lossless-paths.md`, `docs/adr/README.md`

Interfaces: Consumes all merged PRs 1–4.

- [x] **Step 1: Verify GHSA-3pv8-6f4r-ffg2 externally** (`gh api /advisories/GHSA-3pv8-6f4r-ffg2` or WebFetch github.com/advisories). Record affected/fixed tar range. RustSec says tar 0.4.45 already patched RUSTSEC-2026-0067/0068; if the GHSA maps to one of those or to ≤0.4.46, the update below covers it; if the ID is wrong, correct issue #38 with a comment. — evidence: commit 474744b (message records verification against the GitHub advisory DB; not yet mirrored in RustSec)
- [x] **Step 2: Update lockfile** — `cargo update -p tar -p anyhow` (→ tar 0.4.46, anyhow 1.0.104). `cargo audit` → zero vulnerabilities; warnings only number_prefix + paste. — evidence: commit 474744b (PR #51)
- [x] **Step 3: Write `docs/SECURITY.md`** — table: advisory / crate / status (fixed version or unmaintained-documented) / path (`indicatif → number_prefix`, `image → exr|ravif → … → paste`) / disposition and revisit condition. — evidence: commit 474744b (PR #51)
- [x] **Step 4: Add CI audit job:** — evidence: commit 474744b (PR #51; checks:write permission fixed afterwards in PR #52)

```
 audit:
   runs-on: ubuntu-latest
   steps:
     - uses: actions/checkout@v4
     - uses: rustsec/audit-check@v2
       with:
         token: ${{ secrets.GITHUB_TOKEN }}
```

- [x] **Step 5: Bump version** to 1.0.1; run the full gate locally: `cargo test && cargo clippy --all-targets --locked -- -D warnings && cargo fmt --check && cargo build --release && cargo audit`. — evidence: commit 474744b (Cargo.toml 1.0.1) / PR #51 CI green
- [x] **Step 6: Tick the five v1.0.1 checkboxes in docs/ROADMAP.md.** Write the two ADRs (output-safety boundary; master signature + lossless-path JSON representation) following Context/Decision/Alternatives/Consequences. — evidence: ADRs 0001/0002 in commit 474744b; roadmap ticked in commit a2998bb (PR #53 — the roadmap only reached main via PR #42 after the milestone closed)
- [x] **Step 7: Commit + PR** — `chore: remediate advisories, document unmaintained deps, release v1.0.1 (#38)`; `Closes #38`. Merge after CI. — evidence: commit 474744b / PR #51 merged (ac32be1)
- [x] **Step 8: Tag + release** (only now — every gate green): — evidence: PR #51; tag `v1.0.1` and GitHub release "v1.0.1 — Safety baseline" published 2026-07-24, both verified

```
git checkout main && git pull
git tag -a v1.0.1 -m "v1.0.1 — safety baseline"
git push origin v1.0.1
gh release create v1.0.1 --title "v1.0.1 — Safety baseline" --notes-file <(…)
```

Release notes name the five safety fixes and state the archive-immutability invariant.

- [x] **Step 9: Close the milestone**; verify all five parents + children closed; update design-docs/WORKLOG.md; save task summary to ~/projects/_claude-outputs/. — evidence: PR #51; milestone "v1.0.1 — Safety baseline" closed with all 9 issues (#3/#5/#36/#37/#38 + #43–#46) closed, WORKLOG entry 2026-07-23, summary 2026-07-23_backupsage-v1.0.1-safety-baseline_summary.md on disk

---

Self-Review

- **Spec coverage:** #3 → Tasks 1–3 (all five criteria: collisions incl. sidecars/symlink/hardlink Task 2 tests 1–2; foreign/other-source test 4; interrupted preservation test 5; distinct-dest + same-source test 6; archives never write-opened — unchanged read paths + invariant). #36 → Task 4 (aliases, no unlink/truncate, complete report, no partial on failure). #37 → Task 5 (byte-identical rejection + no sidecars, signed open + migration, absent-path creation, alias matrix). #5 → Tasks 6–7 (no raw controls, all sinks routed, JSON round-trip, Unicode stable). #38 → Task 8 (lockfile, audit documentation, gates, notes, tag-last). Child-issue process → Task 0.
- **Placeholder scan:** none — every code step carries real code; Task 6 sink wiring is enumerated by exact file:line list.
- **Type consistency:** `ProtectedSet.check_db_dest` used by indexer (Task 2) and defined in Task 1; `write_new_file(dest, bytes, &ProtectedSet)` signature identical in Tasks 1 and 4; `MasterProbe` variants match between probe and open_at; `path_bytes` hex naming consistent across report.rs/searcher/main.
- **Known risks:** `m.list()` field names (`db_path`/`source_path`) and exact `IndexRun` internals verified at implementation time against current code; `interrupted_reindex` test relies on corrupt-archive failure happening after staging begins (it does — detect_file passes only 512-byte sniff… if sniff rejects garbage before staging, adjust the fixture to a truncated-mid-entry valid-header tar so failure lands mid-build).